



Biome.js

A practical migration and live-demo guide for TypeScript and React teams

React Hook Form working copy
Biome 2.5.13 | 30 minutes

The legacy quality-toolchain problem

Formatting

Prettier establishes stable code layout. It does not make semantic code-quality decisions.

Linting

ESLint plus TypeScript, React, import-sort, and test plugins provide rules. Each plugin adds configuration and upgrade surface.

Developer experience

Editors, hooks, and CI often invoke separate commands. Teams must keep scope, ignores, versions, and behavior aligned.

Biome brings formatter, linter, import organization, assist actions, and a language server into one toolchain.

What Biome provides

Formatter

Formats JavaScript, TypeScript, JSON, CSS, GraphQL, and more. Project settings live in biome.json.

Analyzer

Lint rules and assist actions report diagnostics and can offer safe or explicitly unsafe fixes.

Language server

The editor extension reads the same project configuration used by command-line and CI checks.

Core commands: biome format, biome lint, biome check, biome migrate, biome explain, biome rage

Why teams evaluate Biome

One configuration

A single configuration can align formatting and many lint decisions across developer machines and CI.

Fast feedback

This demo measured formatter checks on the same source scope. Treat the result as local evidence, not a universal benchmark.

Explicit tradeoffs

Biome does not promise full ESLint-plugin or Prettier-output parity. Migration requires rule and formatter review.

Adoption question: does the team accept the formatter output and the remaining rule-coverage boundary?

Biome, Prettier, and ESLint

Question	Prettier	ESLint	Biome
Primary role	Formatting	Lint rules plus plugins	Formatter, linter, assist
Configuration	.prettierrc	eslint.config.mjs	biome.json
Fix command	prettier --write	eslint --fix	biome check --write
Plugin ecosystem	Parser and plugin ecosystem	Largest and most mature	Growing, not complete parity
Type checking	No	Rules may be type-aware	No replacement for tsc

The live demo working copy

Baseline

React Hook Form at commit 287bca28...
f9fee
Existing ESLint 9 flat config, Prettier 3,
TypeScript 6, pnpm lockfile.

Side-by-side scripts

legacy:lint
legacy:format:check
legacy:check
biome:lint
biome:format:check
biome:check

Disposable fixtures

A reset command restores intentionally
failing TSX and syntax examples. Live
writes never touch upstream code.

Working directory: /Users/max/workspace/learning/react-hook-form-biomejs-demo

Live terminal sequence

```
$ pnpm demo:reset  
$ pnpm biome:fixture:check  
$ pnpm biome:fixture:write  
$ git diff -- demo-fixtures/working/FormattingAndLintDemo.tsx  
$ pnpm biome:fixture:write:unsafe  
$ pnpm biome:fixture:syntax  
$ pnpm demo:reset
```

Explain safe fixes first. Review the diff. Then show the optional unsafe template-literal change and the strict parser diagnostic.

A safe migration path

1

Baseline

Commit a clean state.
Keep ESLint and
Prettier active.

2

Preview

Run biome migrate
prettier and biome
migrate eslint without --
write.

3

Reconcile

Accept formatter
settings and select lint
rules deliberately.

4

Advisory CI

Triage output before
making the Biome job
required.

5

Cut over

Retain tsc, tests, and
any ESLint plugins
that still add value.

In this repo, Biome's migration preview reported 54 direct rule migrations from 476 discovered ESLint rules.

Common migration issues and fixes

Formatter differences

Align supported options. Keep an intentional-difference list. Biome documents known divergences from Prettier.

Rule coverage gaps

Use the migration summary. Keep ESLint plugins where they cover rules Biome does not provide.

Configuration and parsing

Check ignored paths, plugin loading, and syntax errors. Biome does not format malformed code by default.

Avoid declaring success from a generated config alone. A successful migration is an accepted behavior change.

Debugging and team integration

Debugging commands

```
biome explain useButtonType  
biome rage --formatter --linter  
biome check --reporter=github  
biome migrate eslint
```

Adoption guardrails

Use the official editor extension and one formatter on save.
Start CI as advisory.
Keep tsc and tests.
Make the check required only after triage.

Biome centralizes many checks. It does not replace every specialized tool or TypeScript type checking.

Decision checklist

- ☐ Formatter output is acceptable for this repository
- ☐ Required ESLint plugin rules have an owner or documented gap
- ☐ Editor, staged-file, and CI behavior agree
- ☐ Type checking and test checks remain independent
- ☐ The team has reviewed a representative migration diff

Recommendation: migrate formatting first, then adopt lint rules incrementally.